KHOA ĐIỆN TỬ VIỄN THÔNG

LẬP TRÌNH ĐA NỀN TẢNG

Chủ đề 5. Danh sách, lưới dữ liệu và cuộn

Flutter: ListView • GridView • Scroll • Lazy loading

Sinh viên thực hiện:
Trần Đăng Duy
Phan Văn Trà

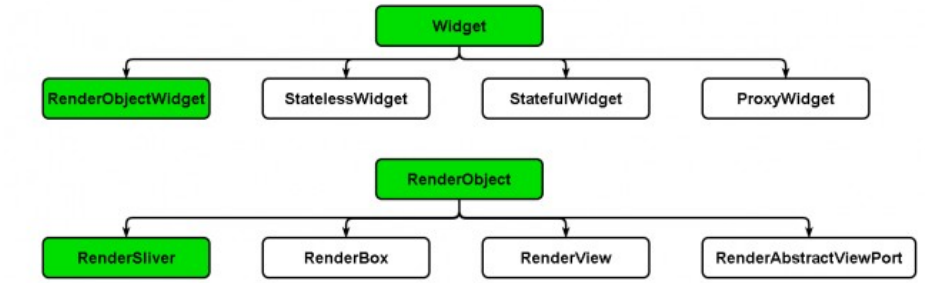
Giảng viên hướng dẫn:
Nguyễn Duy Nhật Viễn

Nội dung trình bày

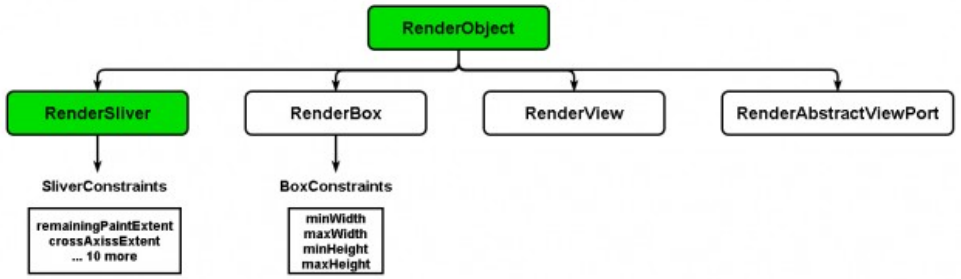
- **1. Cơ sở lý thuyết**
  - Cơ chế cuộn: Viewport & Sliver, nguyên lý bố cục (constraints)
  - Dựng lười: Widget – Element – RenderObject; ListView, GridView
  - ScrollController, ScrollPhysics, lazy loading & phân trang
  - Cuộn lồng nhau, SingleChildScrollView, shrinkWrap
  - Item kích thước khác nhau, trạng thái rỗng, RefreshIndicator
- **2. Thực hành – demo ứng dụng "Thư viện mini"**
  - Chức năng ⇄ kiến thức & cấu trúc project
  - Cuộn lồng nhau, lazy loading, tìm kiếm/rỗng, item co giãn
  - GridView & trang chi tiết (SingleChildScrollView)
- **3. Kết luận & tài liệu tham khảo**

Cơ chế cuộn trong Flutter: Viewport & Sliver

- Mọi widget cuộn (ListView, GridView, CustomScrollView…) đều gồm **Scrollable** + **Viewport**.
- **Scrollable**: nhận cử chỉ (kéo, vuốt), tính vị trí cuộn và quán tính.
- **Viewport**: "ô cửa sổ" chỉ cho nhìn thấy một đoạn của nội dung dài hơn nó.
- Con của Viewport là **Sliver** – một "lát cắt" nội dung biết bố cục theo vùng nhìn thấy.
- Sliver nhận **SliverConstraints** (đang cuộn tới đâu, còn bao nhiêu vùng vẽ…) rồi trả **SliverGeometry** (chiếm bao nhiêu chỗ) ⇒ chỉ dựng phần cần thiết.
- ListView ≈ CustomScrollView + 1 **SliverList**; GridView ≈ CustomScrollView + 1 **SliverGrid**.



Widget / RenderObject: RenderSliver bên cạnh RenderBox

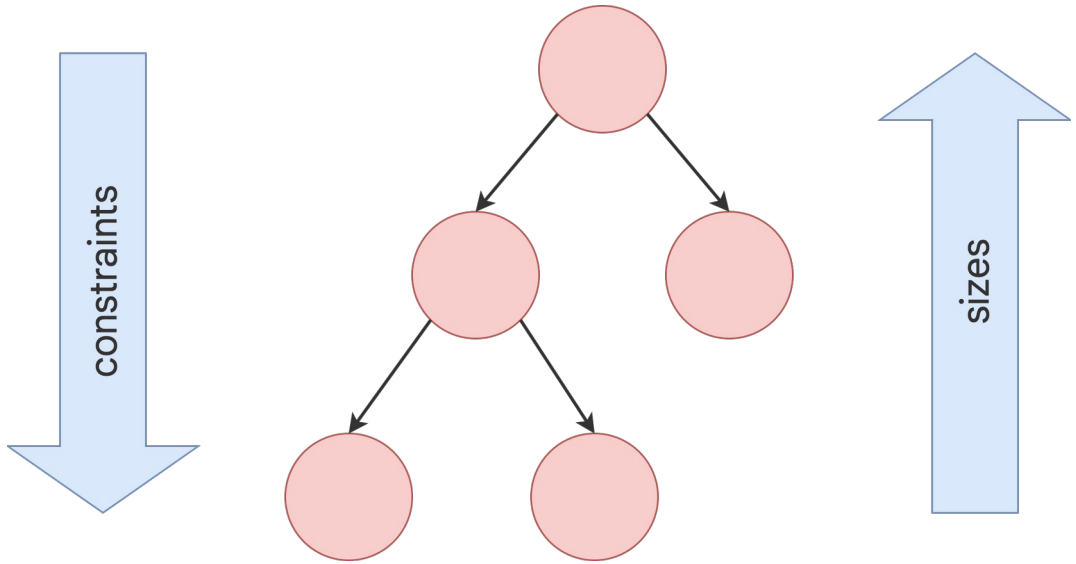


RenderSliver dùng SliverConstraints thay cho BoxConstraints

Nguồn: Kodeco – "Slivers in Flutter: Getting Started" (kodeco.com/19539821) · api.flutter.dev/flutter/widgets/Viewport-class.html

Nguyên lý bố cục: ràng buộc đi xuống, kích thước đi lên

- Bố cục Flutter chạy một lượt trên cây: **cha truyền ràng buộc xuống** (min/max rộng, cao).
- **Con chọn kích thước** trong khoảng cho phép rồi **báo ngược lên** cha.
- **Cha quyết định vị trí** của con; con không tự biết mình nằm đâu.
- Vùng cuộn có **chiều cuộn không giới hạn** (unbounded) ⇒ nội dung có thể dài vô hạn.
- Hệ quả: đặt widget cuộn trong widget cũng không giới hạn chiều (Column, một ListView khác) ⇒ viewport không biết cao bao nhiêu ⇒ lỗi **"Vertical viewport was given unbounded height"**.
- Cách xử lý: cho viewport một **chiều cao xác định** (SizedBox, Expanded).

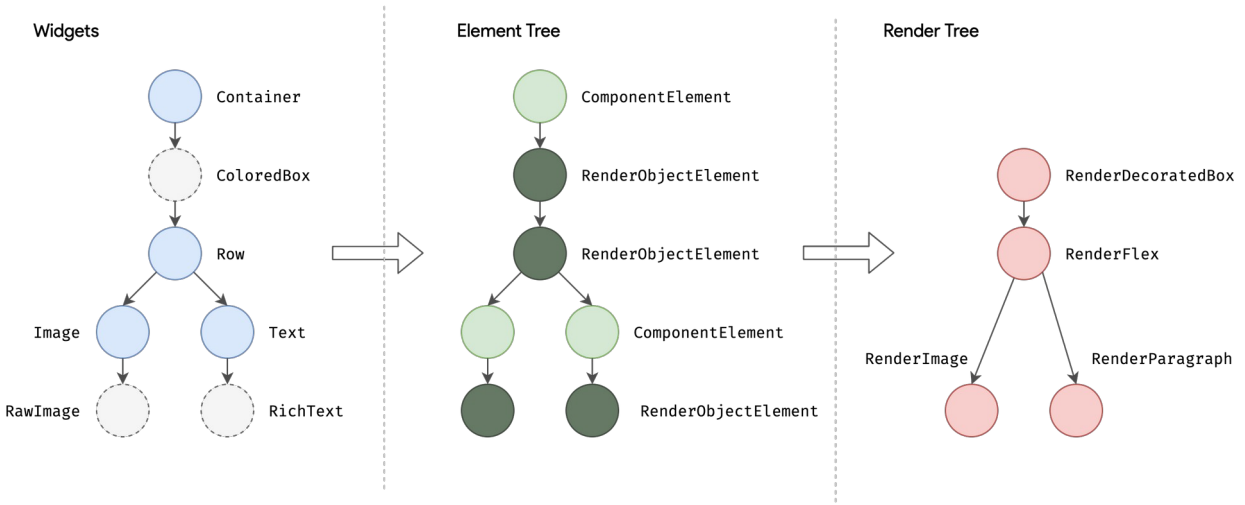


"Constraints go down. Sizes go up. Parent sets position."

Nguồn: docs.flutter.dev/resources/architectural-overview · docs.flutter.dev/ui/layout/constraints

Vì sao dựng lười rẻ? Widget – Element – Render

- Flutter có **3 cây** song song: **Widget** (mô tả cấu hình, bất biến), **Element** (thể hiện, giữ vòng đời), **RenderObject** (đo, bố cục, vẽ).
- Widget rất nhẹ, tạo mới mỗi lần build; **Element/RenderObject được tái sử dụng** khi kiểu widget không đổi.
- Danh sách dùng **builder**: Sliver chỉ yêu cầu delegate tạo widget cho **index đang cần**, rồi huỷ khi ra khỏi vùng.
- ⇒ Số RenderObject sống cùng lúc ~ số item **nhìn thấy**, không phụ thuộc tổng số item (10 hay 10 000).

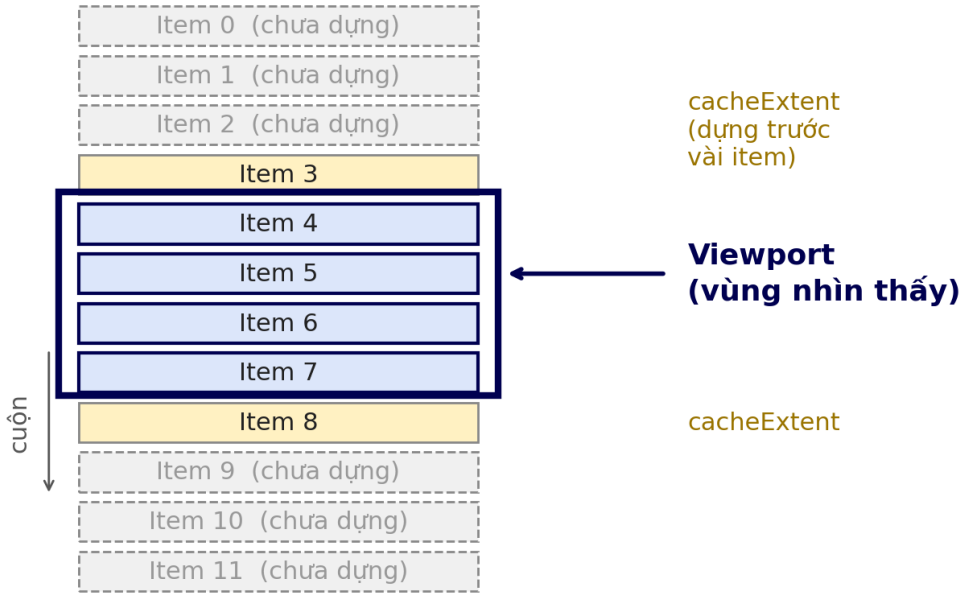


Cây Widget → Element → Render (tài liệu Flutter)

Nguồn: docs.flutter.dev/resources/architectural-overview (Widgets, Elements, RenderObjects)

ListView: dựng toàn bộ so với dựng lười

- **Dựng toàn bộ** (children: […]): tạo mọi item ngay từ đầu ⇒ O(n) về thời gian và bộ nhớ; chỉ hợp khi danh sách **ngắn, cố định**.
- **Dựng lười** (builder): chỉ tạo item **giao với viewport** cộng một vùng đệm **cacheExtent** (mặc định 250 px) để cuộn mượt.
- Item ra khỏi vùng nhìn thấy bị huỷ/tái chế; trạng thái bên trong item không tự giữ (cần KeepAlive nếu muốn).
- Cần biết trước **số lượng** (itemCount) để tính thanh cuộn; nếu không, danh sách coi như vô hạn.
- Biến thể: **separated** (chèn ngăn cách giữa các item), **hướng ngang** (scrollDirection).

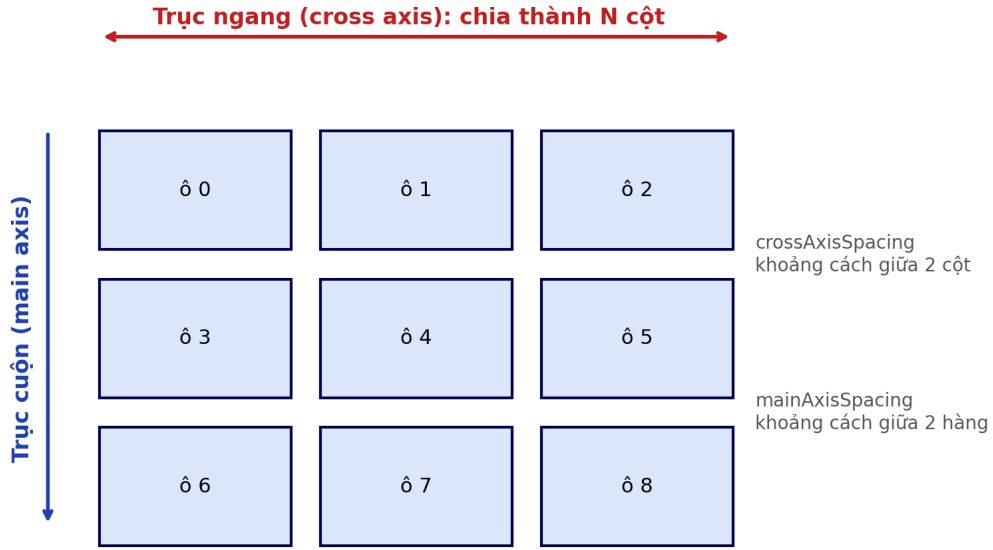


Chỉ item trong viewport (xanh) và vùng đệm (vàng) được dựng

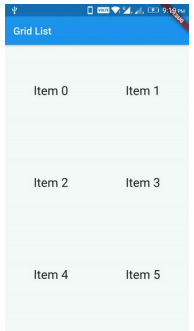
Nguồn: Sơ đồ: nhóm tự vẽ · api.flutter.dev/flutter/widgets/ListView-class.html · docs.flutter.dev/cookbook/lists/long-lists

GridView: nguyên lý chia ô qua SliverGridDelegate

- Lưới cũng là sliver lười; **gridDelegate** quyết định cách **chia trục ngang (cross axis)**; trục cuộn (main axis) kéo dài vô hạn.
- **FixedCrossAxisCount**: cố định N cột ⇒ rộng ô = (bề rộng − khoảng cách × (N−1)) / N.
- **MaxCrossAxisExtent**: mỗi ô rộng tối đa X px ⇒ số cột **tự tính** theo bề rộng màn hình (thích ứng xoay máy, tablet).
- Chiều cao ô = rộng ô / **childAspectRatio**, hoặc đặt cứng bằng **mainAxisExtent**.
- Mọi ô trong một hàng **cùng chiều cao** ⇒ lưới đều, khác với danh sách có item co giãn.



Hai trục của lưới và các khoảng cách

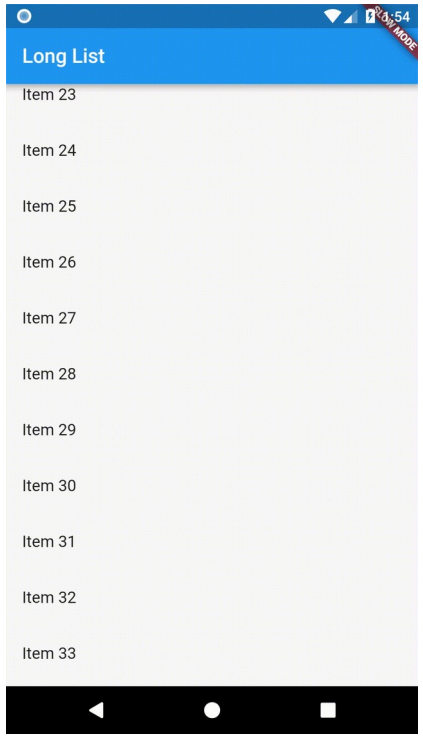


Ví dụ lưới (cookbook)

*Nguồn: Sơ đồ: nhóm tự vẽ · api.flutter.dev/flutter/widgets/GridView-class.html · SliverGridDelegate**

ScrollController, ScrollPosition & ScrollPhysics

- **ScrollPosition**: trạng thái cuộn hiện tại – `pixels` (vị trí), `minScrollExtent`, `maxScrollExtent`, `viewportDimension`.
- **ScrollController**: gắn vào widget cuộn để **đọc/điều khiển** vị trí (`jumpTo`, `animateTo`) và lắng nghe thay đổi; phải **dispose**.
- Cách khác: **NotificationListener<ScrollNotification>** lắng nghe từ ngoài, không cần controller.
- **ScrollPhysics** quyết định "cảm giác" cuộn: **Clamping** (Android – dừng cứng ở mép), **Bouncing** (iOS – nảy lại), **NeverScrollable**, **AlwaysScrollable**.
- Phần còn lại chưa xem ≈ `maxScrollExtent − pixels`.

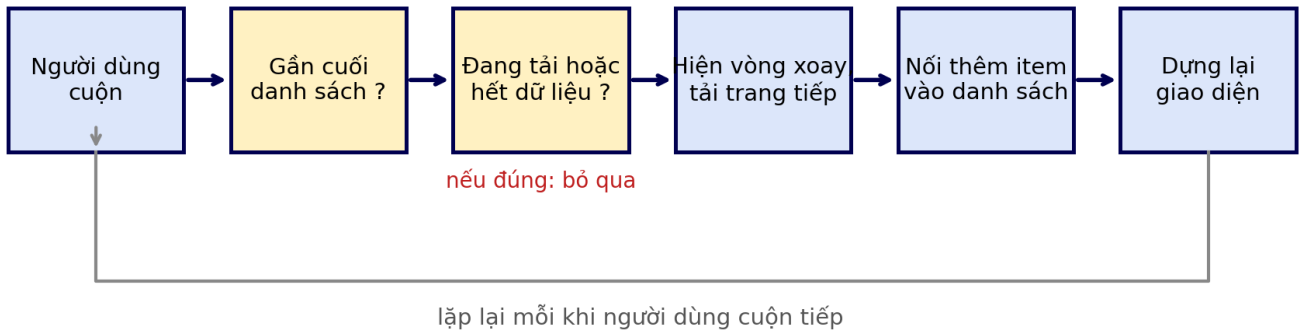


Danh sách dài cuộn trong viewport (cookbook)

Nguồn: api.flutter.dev/flutter/widgets/ScrollController-class.html · ScrollPosition · ScrollPhysics

Lazy loading (infinite scroll) & phân trang

- **Lazy rendering** (dựng item khi cần) khác **lazy loading** (tải dữ liệu khi cần): hai kỹ thuật bổ sung nhau.
- Dữ liệu lớn không nạp một lần: chia **trang** (offset/limit hoặc con trỏ cursor), cuộn gần cuối mới tải trang kế.
- Dùng **ngưỡng** (cách cuối vài trăm px) để tải **trước** khi người dùng chạm đáy ⇒ không bị khựng.
- Cần **cờ trạng thái**: đang tải (tránh gọi trùng) và còn dữ liệu (dừng khi hết).
- UX: hiện **chỉ báo tải** ở cuối danh sách; hết dữ liệu thì thông báo rõ ràng.



Luồng tải thêm khi cuộn

Nguồn: Sơ đồ: nhóm tự vẽ · Viblo: "Lazy load trong Flutter" (viblo.asia/p/lazy-load-trong-flutter-3Q75wE03ZWb)

Cuộn lồng nhau, SingleChildScrollView & shrinkWrap

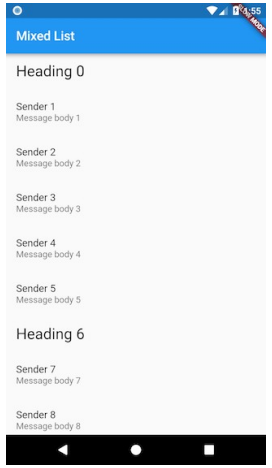
	SingleChildScrollView	ListView.builder	shrinkWrap: true
Cách dựng	Dựng toàn bộ con	Lười, theo viewport	Đo toàn bộ item
Độ phức tạp	O(n)	O(số item hiển thị)	O(n)
Hợp với	Nội dung ngắn, cố định (form, trang chi tiết)	Danh sách dài/không biết trước	Danh sách rất ngắn lồng trong vùng khác
Nhược điểm	Chậm/tốn RAM nếu nhiều con	Cần chiều cao xác định	Mất lợi thế lazy

- **Cuộn lồng nhau khác hướng** (ngang trong dọc): hợp lệ, cần cho widget trong **một chiều cao cố định**.
- **Cuộn lồng cùng hướng**: hai vùng tranh cử chỉ, trải nghiệm xấu ⇒ gộp bằng **CustomScrollView + nhiều Sliver** để chỉ có **một** viewport.
- Nguyên tắc chọn: **ít & cố định** → SingleChildScrollView; **nhiều/lớn** → builder; **nhiều loại khối** → Sliver.

Nguồn: docs.flutter.dev/ui/layout/scrolling/slivers · Medium: "Common mistakes with ListViews in Flutter" (medium.com/@pomis172)

Item kích thước khác nhau, trạng thái rỗng & kéo để làm mới

- **Item kích thước khác nhau**: mỗi item tự đo theo nội dung (văn bản dài ngắn khác nhau); thanh cuộn phải **ước lượng** tổng chiều dài.
- Nếu mọi item bằng nhau, khai báo **itemExtent / prototypeItem** để bỏ qua bước đo ⇒ cuộn nhanh hơn.
- **Trạng thái rỗng (empty state)**: không để màn hình trắng; nêu **lý do** (chưa có dữ liệu, không khớp tìm kiếm) và **hành động tiếp** (xoá bộ lọc, thử lại).
- **Kéo để làm mới (pull-to-refresh)**: người dùng kéo quá mép trên ⇒ gọi tác vụ bất đồng bộ; chỉ báo xoay **đến khi tác vụ hoàn thành**.
- Yêu cầu: con phải là widget cuộn **luôn cuộn được** để nhận được cử chỉ kéo quá mép.



Danh sách có nhiều kiểu item (cookbook)



RefreshIndicator – kéo để làm mới

Nguồn: docs.flutter.dev/cookbook/lists/mixed-list · api.flutter.dev/flutter/material/RefreshIndicator-class.html

App “Thư viện mini”: chức năng ⇄ kiến thức

Chức năng trong app	Kiến thức minh họa
2 tab: Danh sách / Lưới	BottomNavigationBar, IndexedStack
Danh sách sách + tải thêm khi cuộn	ListView.builder, ScrollController
“Sách nổi bật” cuộn ngang trong danh sách dọc	Cuộn lồng nhau
Mô tả dài/ngắn, “Xem thêm / Thu gọn”	Phần tử kích thước khác nhau
Tìm kiếm; không kết quả → EmptyView; kéo để làm mới	Trạng thái rỗng, RefreshIndicator
Lưới sách tự chia cột	GridView.builder
Trang chi tiết sách	SingleChildScrollView

Dữ liệu: assets/data/books_200.csv (200 cuốn) đọc bằng BookRepository (rootBundle + gói csv, cache Future). Không dùng DB/API, chỉ dùng setState.

Nguồn: lib/ : main, models/book, data/book_repository, screens/{home, book_list, book_grid, book_detail}, widgets/{book_tile, book_card, book_cover, empty_view}

Demo 1 – ListView.builder + cuộn lồng nhau


```dart
ListView.builder(
  controller: _controller,
  itemCount: prefixCount + visible + 1,
  itemBuilder: (context, index) {
    if (showFeatured && index == 0) {
      return _FeaturedSection(...);  // ngang
    }
    final i = index - prefixCount;
    if (i < visible) {
      return BookTile(book: results[i]);
    }
    return _isLoadingMore                  // cuối
        ? const CircularProgressIndicator()
        : const Text('Đã hiển thị hết');
  },
)
// _FeaturedSection: SizedBox(height: 220)
//   └ ListView.builder(Axis.horizontal)
```

Một ListView, 3 loại item

- **itemCount = đầu + visible + 1**: dải nổi bật, sách đã hiển thị, ô cuối.
- Dải nổi bật là **ListView ngang** trong `SizedBox(height: 220)` ⇒ cuộn lồng nhau, vẫn lazy.
- Ô cuối: vòng xoay khi đang tải hoặc "Đã hiển thị hết".

Nguồn: lib/screens/book_list_screen.dart

Demo 2 – Lazy loading bằng ScrollController


```dart
void _onScroll() {
  final p = _controller.position;
  if (p.pixels >= p.maxScrollExtent - 200) {
    _loadMore();
  }
}

Future<void> _loadMore() async {
  if (_isLoadingMore ||
      _visibleCount >= _results.length) return;
  setState(() => _isLoadingMore = true);
  await Future.delayed(
      const Duration(milliseconds: 350));
  if (!mounted) return;
  setState(() {
    _visibleCount = min(
        _visibleCount + _pageSize, _results.length);
    _isLoadingMore = false;
  });
}
```

Mỗi lần tải thêm 20 sách

- Còn **200 px** là hết ⇒ `_loadMore()`.
- Cờ `_isLoadingMore` và `_visibleCount >= length` chặn gọi trùng / dừng khi hết dữ liệu.
- `Future.delayed` giả lập thời gian tải trang; sau `await` kiểm tra `mounted`.
- (Code rút gọn; bản đầy đủ có `_generation` chống kết quả cũ.)

Nguồn: lib/screens/book_list_screen.dart

Demo 3 – Item co giãn, tìm kiếm & trạng thái rỗng


```dart
// BookTile: chiều cao item thay đổi
Text(book.description,
  maxLines: _expanded ? null : 2,
  overflow: _expanded
      ? TextOverflow.visible
      : TextOverflow.ellipsis);
TextButton(
  onPressed: () =>
      setState(() => _expanded = !_expanded),
  child: Text(_expanded ? 'Thu gọn' : 'Xem thêm'));

// Tìm kiếm → rỗng
if (results.isEmpty) {
  return _statusList(
    message: 'Không tìm thấy sách',
    action: 'Xóa tìm kiếm',
    onPressed: _clearQuery);
}
```

Ba ý chính

- Mô tả dài/ngắn ⇒ **chiều cao item khác nhau**; mặc định 2 dòng, "Xem thêm" mở hết.
- `searchBooks` lọc theo tên/tác giả; đổi từ khoá ⇒ về 20 phần tử đầu, `jumpTo(0)`.
- Không có kết quả ⇒ **EmptyView** + nút "Xóa tìm kiếm".

Nguồn: lib/widgets/book_tile.dart · lib/screens/book_list_screen.dart · lib/data/book_repository.dart

Demo 4 – GridView.builder tự chia cột


```dart
LayoutBuilder(
  builder: (context, constraints) {
    final w = constraints.maxWidth;
    final columns =
        w < 350 ? 1 : max(2, ((w + 8) / 190).floor());
    return GridView.builder(
      gridDelegate:
          SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: columns,
        mainAxisExtent: 250,
        crossAxisSpacing: 8,
        mainAxisSpacing: 8,
      ),
      itemCount: books.length,
      itemBuilder: (context, index) =>
          BookCard(book: books[index]),
    );
  },
)
```

Lưới thích ứng bề rộng

- `LayoutBuilder` đọc bề rộng ⇒ tính **số cột** (< 350 px: 1 cột, còn lại ~190 px/cột).
- `mainAxisExtent: 250` cố định chiều cao ô ⇒ lưới đều.
- Dữ liệu nạp bằng **FutureBuilder** (tải / lỗi / rỗng / có dữ liệu).

Nguồn: lib/screens/book_grid_screen.dart

Demo 5 – Trang chi tiết: SingleChildScrollView


```dart
Scaffold(
  appBar: AppBar(title: Text(book.title)),
  body: SingleChildScrollView(
    child: Center(
      child: ConstrainedBox(
        constraints:
            const BoxConstraints(maxWidth: 720),
        child: Column(children: [
          SizedBox(width: 180, height: 250,
              child: BookCover(book: book)),
          Text(book.title, ...),
          Text('Tác giả: ${book.authorNames}'),
          Text(book.description, ...), // dài
        ]),
      ),
    ),
  ),
)
```

Một khối nội dung, độ dài thay đổi

- Ảnh + vài dòng thông tin + mô tả dài ⇒ `SingleChildScrollView` + `Column`.
- Màn hình nhỏ/mô tả dài vẫn cuộn, không bị **overflow**.
- Hợp lý vì nội dung ít, cố định; không dùng cho danh sách dài.

Nguồn: lib/screens/book_detail_screen.dart

Kết luận & tài liệu tham khảo

- **builder** (ListView/GridView) dựng lazy ⇒ xử lý tốt dữ liệu lớn.
- **ScrollController** phát hiện cuối danh sách để tải thêm.
- **SingleChildScrollView** cho nội dung ngắn; **shrinkWrap** làm mất lazy.
- App minh họa đủ: cuộn lồng nhau, item co giãn, rỗng, làm mới, lưới, chi tiết.

- docs.flutter.dev/cookbook/lists (long-lists, grid-lists, horizontal-list)
- docs.flutter.dev/ui/layout/scrolling/slivers · docs.flutter.dev/resources/architectural-overview
- Kodeco: Slivers in Flutter: Getting Started (kodeco.com/19539821)
- api.flutter.dev: ListView, GridView, SingleChildScrollView, ScrollController, RefreshIndicator
- Viblo: Lazy load trong Flutter (viblo.asia/p/lazy-load-trong-flutter-3Q75wE03ZWb)
- Viblo: Load more và pull to refresh với GridView (viblo.asia/p/jvEla3vdKkw)
- Medium: Flutter: Lazy Loading on Scroll Made Simple (@archelangelo)
- Medium: Common mistakes with ListViews in Flutter (@pomis172)
- Ảnh: flutter.github.io/assets-for-api-docs · docs.flutter.dev/assets/images

CẢM ƠN THẦY VÀ CÁC BẠN

Xin mời đặt câu hỏi – Q&A

Trần Đăng Duy • Phan Văn Trà